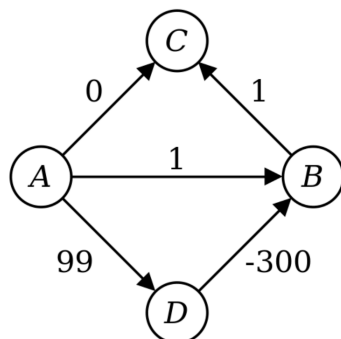


Final Review - 5/6/2026

Name all the ways you could implement a hash table that we have covered in this class (including import options) describe briefly how implementation would change for collision resolution methods and the consequences of changing these methods.

- **Separate Chaining:** A bound by linked list efficiency in the worst case scenario, no secondary collisions or clustering
- **Linear probing:** Limited by the size of the hash table, clustering effects due to collisions
- **Quadratic Probing:** Limited by the size of hash table, and primary clustering is avoided, but secondary clustering can still occur
- **Double Hashing:** Reduce clustering by using an entirely new hashing system
- We can implement a HashTable using chaining with linked lists, there are collisions appended to a chain and its time complexity is $O(n)$ in the worst case. Chaining with a BST improves the look up to $O(\log n)$ but adds balancing. We can use Java's HashMap and HashSet as import options since they use chaining

Observe the following graph.



Can we perform Dijkstra's Algorithm on this graph? Why or why not?

- We cannot do Dijkstra's Algorithm because it has a negative weight. With negative weight we can end up with a not optimal distance. Also Dijkstra's algorithm here might not reach every node. We have one way pointers, and in case we go to node C, we would be stuck there

Write a program that solves the activity selection problem using a greedy algorithm.

You are given a list of activities, where each activity has a name, a start time, and an end time. Your goal is to select the maximum number of activities that can be completed without overlapping.

Two activities are compatible if one activity ends before or exactly when the next activity begins.

```

import java.util.Arrays;

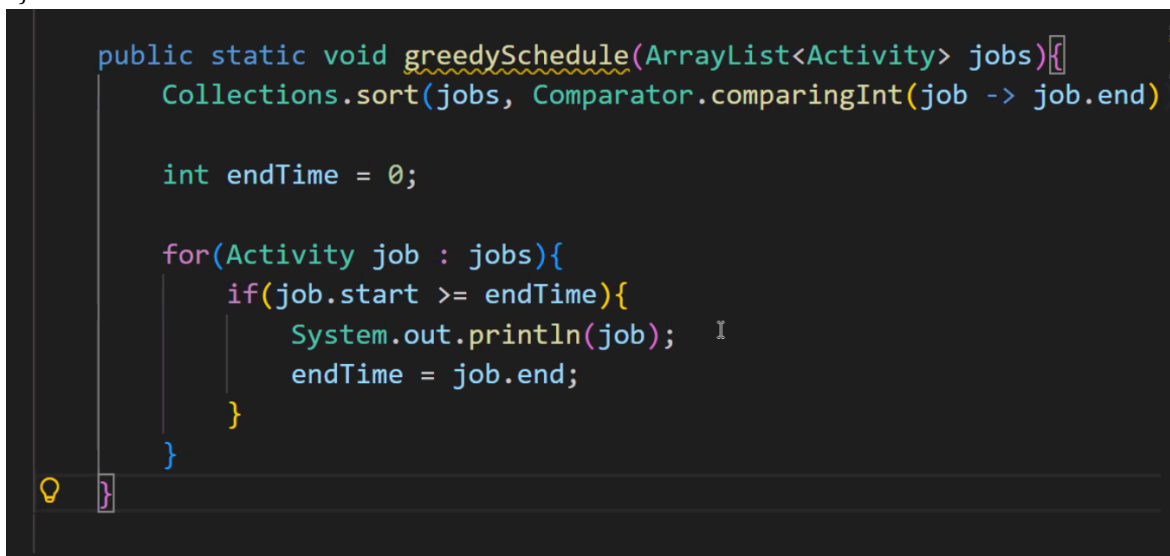
public class Activity {
    public static void main(String[] args) {
        String[] names = {"A", "B", "C"};
        int[] start = {1, 3, 0};
        int[] end = {4, 5, 6};

        Integer[] indices = {0, 1, 2, 3, 4};
        Arrays.sort(indices, (a, b) -> end[a] - end[b]);

        System.out.println("Selected activities:");
        int lastEnd = -1;

        for (int i : indices) {
            if (start[i] >= lastEnd) {
                System.out.println(names[i] + " (start: " + start[i] + ", end: " + end[i] + ")");
                lastEnd = end[i];
            }
        }
    }
}

```



```

public static void greedySchedule(ArrayList<Activity> jobs){
    Collections.sort(jobs, Comparator.comparingInt(job -> job.end)

    int endTime = 0;

    for(Activity job : jobs){
        if(job.start >= endTime){
            System.out.println(job);
            endTime = job.end;
        }
    }
}

```

Write a program that finds the largest sum you can make using numbers from a list, but you are not allowed to use two numbers that are next to each other. For example, given this list: [3, 2, 5, 10, 7] the largest sum would be $3 + 5 + 7 = 15$

```

int[] nums = new int[]{3, 2, 5, 10, 7};

int include = 0;
int exclude = 0;

for (int i = 0; i < nums.length; i++) {
    int newInclude = exclude + nums[i];
    int newExclude = Math.max(include, exclude);
    include = newInclude;
    exclude = newExclude;
}

int largest = Math.max(include, exclude);

```

```

60
61     public static int maxSum(int[] numbers){
62         if(numbers.length == 0){
63             return 0;
64         }
65         if(numbers.length == 1){
66             return numbers[0];
67         }
68
69         int[] tab = new int[numbers.length];
70         tab[0] = numbers[0];
71         tab[1] = Math.max(numbers[0], numbers[1]);
72         for(int i = 2; i < numbers.length; i++){
73             int tempInclude = numbers[i] + tab[i-2];
74             int skipInclude = tab[i-1];
75
76             tab[i] = Math.max(tempInclude, skipInclude);
77         }
78
79         return tab[numbers.length-1];
80     }

```

Given the following code snippet, determine its Big O time complexity and justify your answer:

```
def process(data):  
    for i in range(len(data)):  
        for j in range(i, len(data)):  
            print(data[i], data[j])  
    for k in range(len(data)):  
        print(data[k])
```

- I believe the Big O time complexity is $O(n^2)$. The code has nested loops, so the inner loop runs a number of times depending on i . So it would result in a quadratic complexity

I see that there is also another loop which is separated from the nested loop, that one since it is by itself it runs in linear time which makes the time complexity be $O(n)$

Write a recursive function in Java that takes a string and a character as input and returns the number of times the character appears in the string. Then, explain the base case and the recursive case of your function in plain text underneath your code submission.

Example Input/Output:

`count_occurrences("banana", "a")` # Output: 3

`count_occurrences("recursion", "r")` # Output: 2

`count_occurrences("hello", "z")` # Output: 0

`count_occurrences("", "a")` # Output: 0

Requirements:

- Your function should be fully recursive (no loops allowed).
- It should have a clear base case and recursive case.

```

public static int characterInString(String word, char character) {
    if (word.length() == 0) {
        return 0;
    }
    if (word.charAt(0) == character) {
        return 1 + characterInString(word.substring(1), character);
    } else {
        return 0 + characterInString(word.substring(1), character);
    }
}

```

- The base case for this is checking if the length of the word is equal to 0; if it is 0, which would be empty, then the program would return 0, and it stops. The recursive cases are checking the first character and then calling the function again on the remainder of the string, which is the (substring(1)). So it breaks the big task into some smaller versions of the task, checking until the string is empty to return the count.

Balanced AVL Trees are allowed to skew in one direction or another.

- True

Which of the following are true about AVL trees and Binary Search Trees as we have implemented in class?

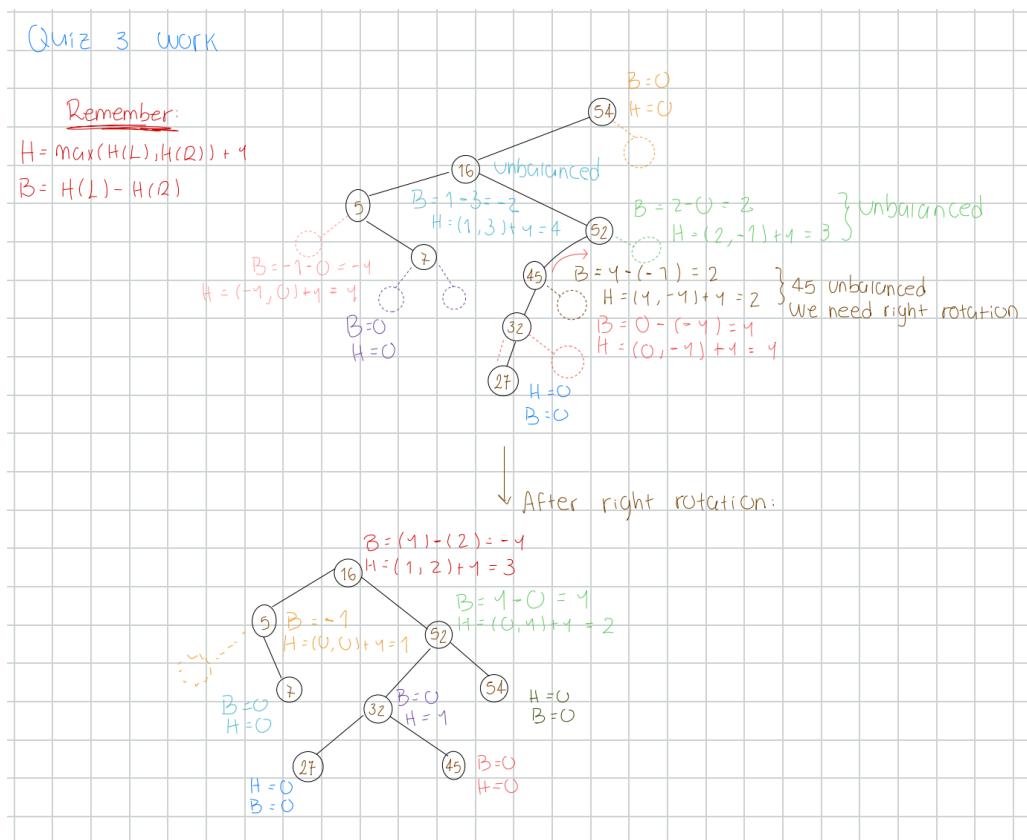
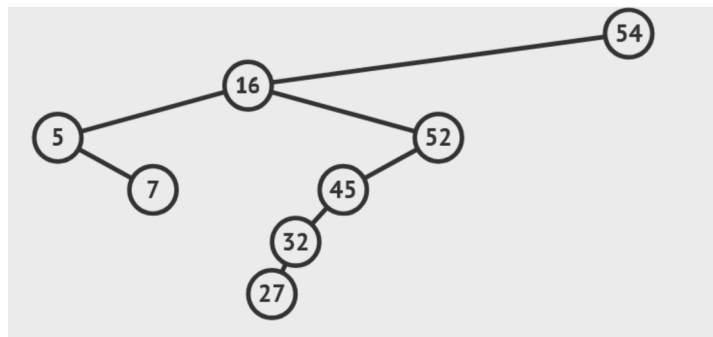
AVL trees allow duplicate values and BSTs don't

AVL trees store height information at the leaves only

Every AVL tree is also a BST

AVL Trees guarantee $O(\log n)$ efficiency on search

Using pen, pencil, or the paint program, caculate the points of imbalance in the tree below as you go through and perform rotations following AVL tree rules. Show all intermediate trees with the next point of imbalance pointed out with its balance value. Upload a picture of your final work.



In java, write a function that checks for duplicates within an array that has linear time-efficiency.

```

import java.util.Array;
import java.util.HashSet;

public static boolean duplicates (int [] array) {
    HashSet<Integer> checked = new HashSet<>();
  
```

```
for (int i = 0; i < array.length; i++) {  
    if (checked.contains(array[i])) {  
        return true;  
    }  
    checked.add(array[i]);  
}  
return false;  
}
```